

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
Высшего профессионального образования
«Казанский (Приволжский) Федеральный университет»

**ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ**

Кафедра прикладной математики и искусственного интеллекта

ОТЧЕТ

Многослойный персептрон

Студент 3 курса
группы 09-321
С.С. Чуркин

«_____» _____ 2026 г.

Содержание

1	Введение	2
2	Постановка задачи	2
3	Ход работы	3
3.1	Анализ и предварительная обработка данных	3
3.1.1	Характеристики датасета	3
3.1.2	Заполнение пропущенных данных	3
3.1.3	Нормализация признаков	3
3.2	Визуализация данных	4
3.2.1	Баланс классов	4
3.2.2	Гистограммы распределения признаков	4
3.2.3	Корреляционный анализ и выявление мультиколлинеарности	5
3.3	Реализация многослойного персептрона	7
3.3.1	Архитектура сети и прямой проход	7
3.3.2	Функция потерь и обратное распространение ошибки	7
3.3.3	Градиентный спуск и алгоритмы оптимизации	8
3.4	Обучение и результаты	8
3.4.1	Разделение выборки	8
3.4.2	Кривые обучения	8
3.4.3	Оценка качества классификации	9
3.4.4	Сравнительный анализ алгоритмов оптимизации	9
4	Выводы	10

1 Введение

В настоящее время искусственные нейронные сети являются одним из ключевых инструментов решения задач распознавания образов и классификации данных. Одним из фундаментальных классов нейронных сетей является многослойный персептрон (Multilayer Perceptron, MLP), способный аппроксимировать сложные нелинейные разделяющие поверхности благодаря последовательному применению взвешенного суммирования и нелинейных функций активации.

В рамках данной лабораторной работы рассматривается прикладная медицинская задача: классификация клеток опухолей молочной железы на доброкачественные (Benign) и злокачественные (Malignant). Данная задача имеет критическое значение в практической медицине, где ключевым критерием качества является минимизация ошибок второго рода (пропуск злокачественного образования).

Основное дидактическое требование работы заключается в реализации всей математики многослойного персептрона с нуля с использованием языка программирования Python и библиотеки линейной алгебры NumPy, без привлечения высокоуровневых библиотек глубокого обучения (PyTorch, TensorFlow, Keras, scikit-learn).

2 Постановка задачи

В соответствии с техническим заданием необходимо выполнить следующие этапы:

1. Анализ и предварительная обработка данных:

- Исследовать набор данных `data.csv` (Wisconsin Diagnostic Breast Cancer), состоящий из 569 строк и 32 столбцов.
- Проверить наличие пропущенных значений и реализовать логику их заполнения на основе условного среднего по соответствующему классу.
- Выполнить нормализацию числовых признаков для обеспечения устойчивой сходимости градиентного спуска.

2. Разработка структуры программных модулей:

- `split.py` – программа разделения исходного набора данных на обучающую (Train, 80%) и проверочную (Validation, 20%) части со стратификацией и фиксированным значением `seed`.
- `train.py` – программа обучения многослойного персептрона, использующая алгоритм обратного распространения ошибки (Backpropagation) и градиентный спуск, с выводом метрик по эпохам, визуализацией кривых обучения и сохранением весов модели.
- `predict.py` – программа прогнозирования, загружающая обученные веса, вычисляющая вероятность принадлежности к классам и оценивающая результат с помощью бинарной кросс-энтропии и точности.

3. Требования к архитектуре сети:

- Входной слой: 30 числовых характеристик ядра клетки.
- Не менее двух скрытых слоев по умолчанию (например, по 24 нейрона в каждом).

- Выходной слой: 2 нейрона с функцией активации Softmax для формирования вероятностного распределения по классам.
- Возможность модульной настройки гиперпараметров (число слоев, нейронов, шаг обучения, размер батча, выбор оптимизатора).

3 Ход работы

3.1 Анализ и предварительная обработка данных

3.1.1 Характеристики датасета

Датасет `data.csv` содержит информацию о 569 образцах клеток биопсии. В структуре данных:

- Столбец 0: уникальный идентификатор пациента (`id`), не несущий полезной диагностической информации и исключаемый из обучения;
- Столбец 1: метка диагноза (`diagnosis`), принимающая значения **B** (доброкачественная, 0) или **M** (злокачественная, 1);
- Столбцы 2–31: 30 числовых характеристик клеточного ядра (радиус, текстура, периметр, площадь, гладкость, компактность, вогнутость, точки вогнутости, симметрия, фрактальная размерность в трех вариантах расчета: mean, standard error, worst).

3.1.2 Заполнение пропущенных данных

В проекте реализован механизм подбора и заполнения пропущенных значений по среднему значению того класса, к которому относится объект (*Class-conditional mean imputation*):

$$\mu_{c,j} = \frac{1}{|S_c|} \sum_{i \in S_c} x_{i,j}, \quad c \in \{0, 1\}, \quad (1)$$

где S_c – множество объектов обучающей выборки, принадлежащих классу c , а j – номер признака.

Логика данного подхода заключается в том, что физические размеры злокачественных клеток существенно отличаются от нормальных клеток. Заполнение общим средним смещало бы истинные границы распределений. Если пропуск обнаруживается у объекта класса **B**, подставляется среднее по классу **B**, а если у объекта **M** – среднее по классу **M**.

На этапе прогнозирования новых данных (`predict.py`), когда истинный класс образца априори неизвестен, для заполнения пропусков применяется глобальное среднее обучающей выборки $\mu_{\text{global},j}$.

3.1.3 Нормализация признаков

В исходных данных признаки имеют принципиально разный порядок величин: значение признака гладкости (`smoothness_mean`) лежит в интервале от 0.05 до 0.16, тогда как площадь (`area_worst`) достигает значений свыше 2500.

Если подать признаки в исходном масштабе, веса, связанные с площадью, будут обновляться с колоссальными градиентами, вызывая расхожимость алгоритма или требуя исчезающе малого шага обучения. Для устранения этой проблемы применяется Z-score стандартизация:

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}, \quad (2)$$

где μ_j и σ_j – выборочное среднее и стандартное отклонение j -го признака.

Важный аспект: параметры μ_j и σ_j вычисляются строго на обучающей выборке (Train) и затем применяются к проверочной выборке (Validation/Test) без пересчета. Это исключает утечку данных (*Data Leakage*).

3.2 Визуализация данных

3.2.1 Баланс классов

Распределение целевой переменной показано на рисунке 1:

- Класс В (Доброкачественные): 357 образцов (62.7%);
- Класс М (Злокачественные): 212 образцов (37.3%).

Дисбаланс является умеренным, однако требует обязательной стратификации при разбиении выборки.

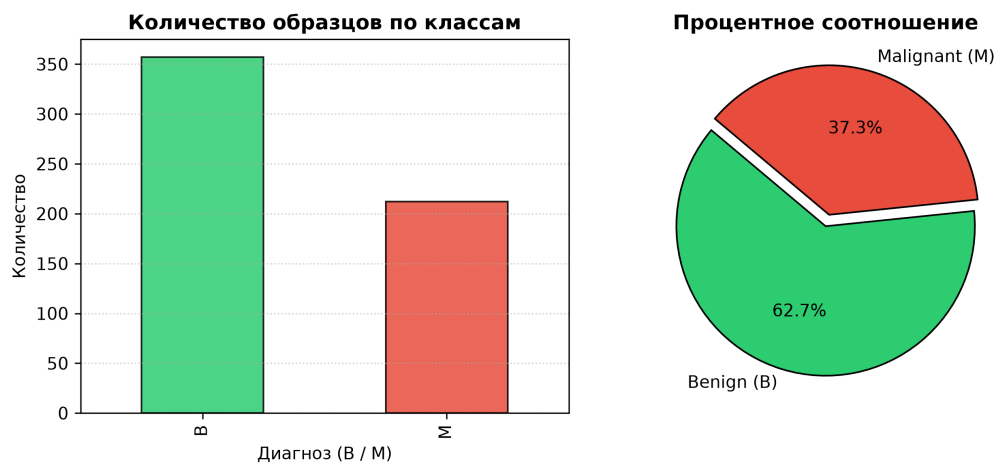


Рис. 1: Распределение классов доброкачественных и злокачественных опухолей.

3.2.2 Гистограммы распределения признаков

На рисунке 2 представлены распределения ключевых характеристик ядер клеток с разделением по диагнозу.

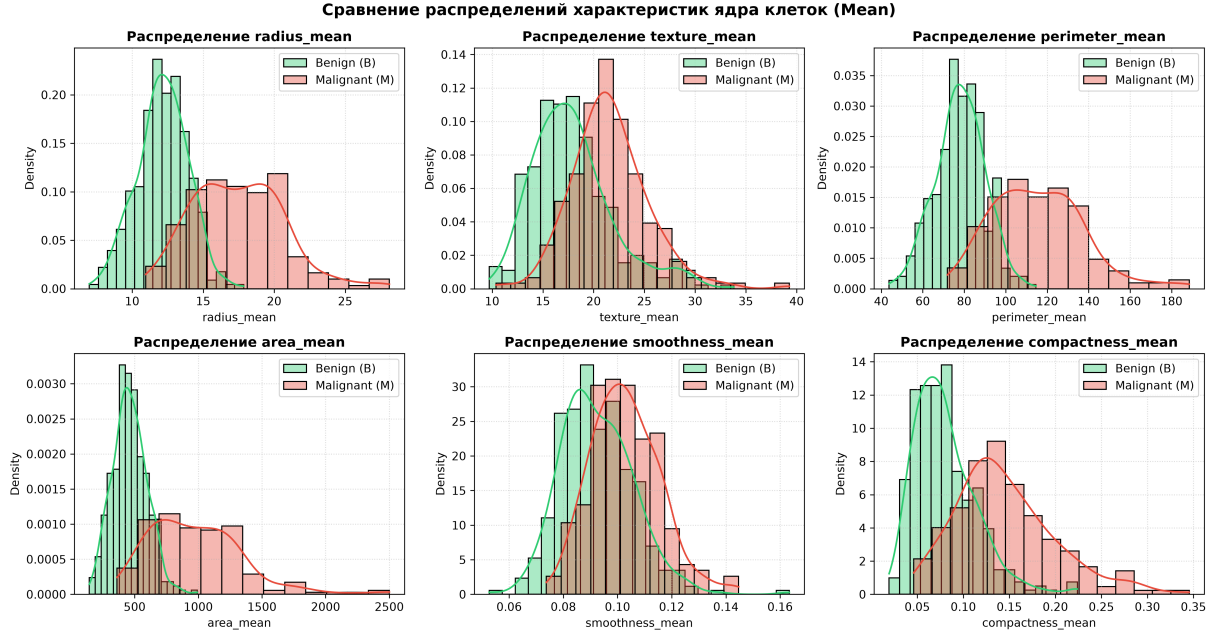


Рис. 2: Сравнение распределений характеристик ядра клеток для классов В и М.

Из графиков видно, что средний радиус (+43.8%), периметр (+47.8%), площадь (+111.4%) и вогнутость (+243%) ядер злокачественных клеток значительно превосходят аналогичные параметры здоровых клеток, что обеспечивает хорошую разделимость классов линейными и нелинейными границами.

3.2.3 Корреляционный анализ и выявление мультиколлинеарности

Корреляционный анализ проводился для оценки степени линейной взаимосвязи между числовыми характеристиками клеточных ядер и выявления избыточной (дублирующей) информации. Для каждой пары признаков X и Y вычислялся коэффициент линейной корреляции Пирсона:

$$r_{XY} = \frac{\sum_{i=1}^m (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^m (x_i - \bar{x})^2 \sum_{i=1}^m (y_i - \bar{y})^2}}. \quad (3)$$

На рисунке 3 приведена тепловая карта корреляции признаков группы mean.

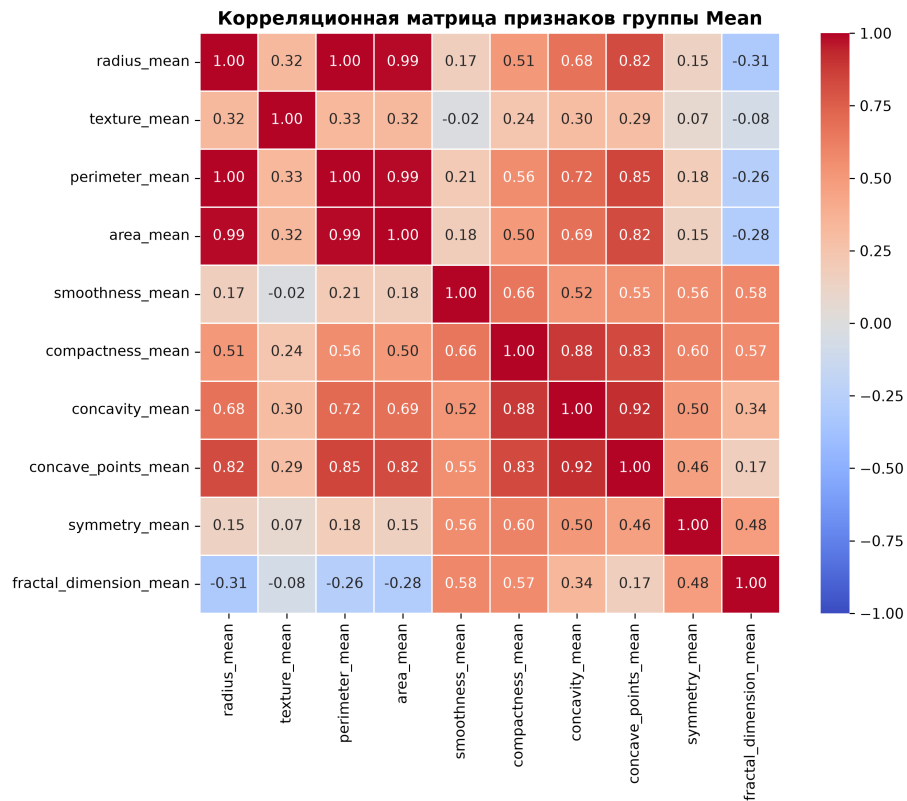


Рис. 3: Корреляционная матрица признаков группы mean.

Анализ полученной матрицы позволил сделать следующие важные выводы:

1. **Высокая мультиколлинеарность геометрических размеров:** Признаки радиуса (`radius_mean`), периметра (`perimeter_mean`) и площади (`area_mean`) обладают почти абсолютной положительной корреляцией ($r \geq 0.98$). Это физически закономерно: периметр и площадь клеточного ядра напрямую зависят от его радиуса ($P \approx 2\pi R$, $S \approx \pi R^2$). С точки зрения машинного обучения эти три признака несут практически идентичную информацию о размере клетки.
2. **Связь параметров вогнутости:** Признаки вогнутости ядра (`concavity_mean`) и числа точек вогнутости (`concave_points_mean`) также тесно скоррелированы между собой ($r \approx 0.92$), так как оба показателя характеризуют степень вдавливания и неправильности контура мембраны.
3. **Независимые информативные признаки:** В отличие от размера, признак текстуры (`texture_mean`, вариация интенсивности серого цвета) и признак гладкости (`smoothness_mean`) слабо скоррелированы с радиусом ($r < 0.35$). Это означает, что они несут уникальную, не дублирующуюся информацию о внутренней плотности хроматина и неоднородности структуры ядра.

Для нейронной сети: хотя многослойный персептрон за счет нелинейных функций активации способен обучаться на скоррелированных данных, наличие сильной мультиколлинеарности приводит к тому, что градиенты функции потерь по дублирующим весам делятся между собой, делая поверхность ошибки вытянутой. Понимание взаимосвязей признаков позволяет обосновать работу модели: классификатор опирается не на один изолированный признак, а на три независимых физических аспекта — геометрический размер ядра, степень деформации его границ и внутреннюю текстуру клетки.

3.3 Реализация многослойного персептрона

3.3.1 Архитектура сети и прямой проход

В качестве базовой модели принята полносвязная нейронная сеть следующей конфигурации:

- Входной слой: 30 нейронов (по числу признаков);
- Скрытый слой 1: 24 нейрона, функция активации Sigmoid;
- Скрытый слой 2: 24 нейрона, функция активации Sigmoid;
- Выходной слой: 2 нейрона, функция активации Softmax.

Для каждого слоя l прямое распространение вычисляется по формулам:

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}, \quad (4)$$

$$A^{[l]} = \sigma(Z^{[l]}), \quad \text{где } \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (5)$$

На выходном слое применяется функция Softmax для преобразования пред-активаций $Z^{[L]}$ в вероятности принадлежности к классам 0 (В) и 1 (М):

$$\hat{y}_{ik} = \frac{\exp(z_{ik} - \max_j z_{ij})}{\sum_{j=1}^2 \exp(z_{ij} - \max_j z_{ij})}, \quad k \in \{1, 2\}. \quad (6)$$

Вычитание максимума в строке обеспечивает численную устойчивость и защищает от переполнения типа `float64`.

3.3.2 Функция потерь и обратное распространение ошибки

В качестве целевой функции минимизации используется бинарная кросс-энтропия (Binary Cross-Entropy):

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^2 y_{ik} \ln(\hat{y}_{ik} + \epsilon), \quad (7)$$

где y_{ik} – истинная метка в формате One-Hot ($[1, 0]$ для В и $[0, 1]$ для М), а $\epsilon = 10^{-15}$ защищает от вычисления $\ln(0)$.

При совместном использовании функции Softmax и функции потерь Cross-Entropy производная по логитам выходного слоя принимает предельно компактный вид:

$$dZ^{[L]} = \hat{Y} - Y. \quad (8)$$

Обратное распространение ошибки по предыдущим слоям вычисляется по цепному правилу (Chain Rule):

$$dW^{[l]} = \frac{1}{m} (A^{[l-1]})^T dZ^{[l]}, \quad db^{[l]} = \frac{1}{m} \sum_{i=1}^m dZ_{i,:}^{[l]}, \quad (9)$$

$$dA^{[l-1]} = dZ^{[l]} (W^{[l]})^T, \quad (10)$$

$$dZ^{[l-1]} = dA^{[l-1]} \odot \sigma'(Z^{[l-1]}), \quad \text{где } \sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (11)$$

3.3.3 Градиентный спуск и алгоритмы оптимизации

Для обновления весов сети в коде реализованы различные алгоритмы оптимизации:

1. **SGD (Stochastic Gradient Descent)** – классический мини-батч градиентный спуск:

$$W \leftarrow W - \alpha \cdot dW, \quad b \leftarrow b - \alpha \cdot db. \quad (12)$$

Базовый алгоритм, в котором шаг пропорционален градиенту функции потерь по текущему мини-батчу.

2. **Momentum (метод тяжелого шарика)** – градиентный спуск с накоплением импульса:

$$v_W \leftarrow \beta v_W + \alpha \cdot dW, \quad W \leftarrow W - v_W. \quad (13)$$

Параметр инерции $\beta \approx 0.9$ накапливает скорость предыдущих шагов, что ускоряет спуск вдоль протяженных оврагов и сглаживает поперечные колебания.

3. **Adam (Adaptive Moment Estimation)** – адаптивный градиентный спуск: алгоритм вычисляет адаптивную скорость обучения для каждого веса индивидуально, объединяя преимущества накопления импульса (скорости) и нормировки на амплитуду градиентов, что обеспечивает быстрое преодоление начального плато и устойчивую сходимость.

3.4 Обучение и результаты

3.4.1 Разделение выборки

Скрипт `split.py` разделил исходный датасет со стратификацией на две части (`seed = 42`):

- Обучающий набор (`data_training.csv`): 456 образцов (286 В, 170 М);
- Проверочный набор (`data_validation.csv`): 113 образцов (71 В, 42 М).

3.4.2 Кривые обучения

Модель была обучена с такими параметрами:

- Архитектура: 2 скрытых слоя по 24 нейрона;
- Скорость обучения $\alpha = 0.0314$;
- Размер батча: 8;
- Число эпох: 84;
- Функция потерь: `binaryCrossEntropy`.

На рисунке 4 представлены кривые обучения (Loss и Accuracy) на обучающей и проверочной выборках.

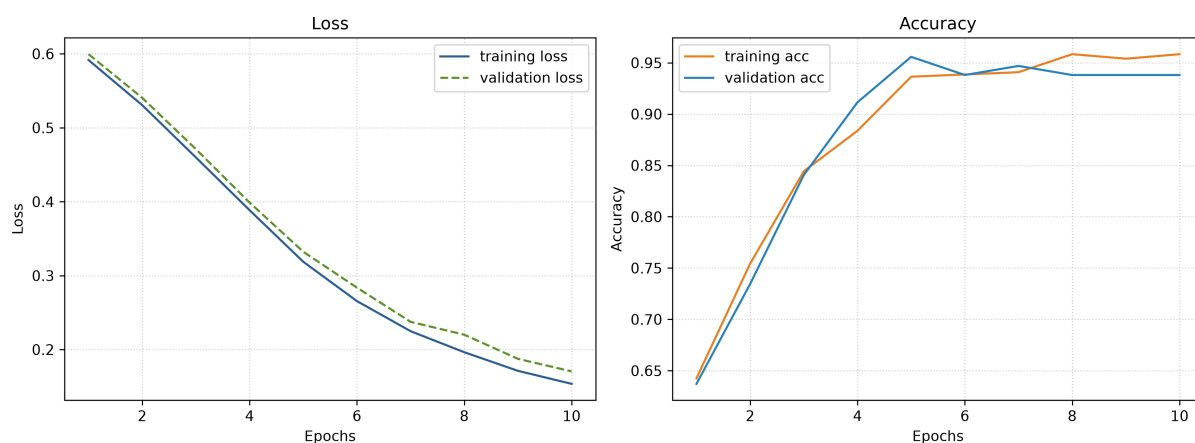


Рис. 4: Кривые обучения многослойного персептрона: функция потерь (слева) и точность (справа).

По графикам видно, что функция потерь монотонно снижается с 0.5508 до 0.0464 на обучающей выборке и с 0.5634 до 0.0931 на проверочной. Отсутствие расхождения кривых валидации свидетельствует об отсутствии эффекта переобучения.

3.4.3 Оценка качества классификации

Оценка модели на независимом проверочном наборе данных с помощью скрипта `predict.py` показала следующие результаты:

Таблица 1: Метрики качества классификации на проверочном наборе данных.

Метрика	Значение
Binary Cross-Entropy Loss	0.0931
Accuracy (Общая точность)	95.58% (108 / 113)
Precision (Точность для Malignant)	91.11%
Recall (Полнота для Malignant)	97.62%
Specificity (Специфичность для Benign)	94.37%
F1-Score	0.9425

Матрица ошибок (Confusion Matrix):

	Предсказано Benign (0)	Предсказано Malignant (1)
Истинно Benign (0)	67 (TN)	4 (FP)
Истинно Malignant (1)	1 (FN)	41 (TP)

В медицинской диагностике наибольшую опасность представляет ошибка второго рода (False Negative, FN) – пропуск заболевания. Наша модель из 42 пациентов со злокачественными опухолями безошибочно выявила 41 (Полнота Recall = 97.62%), допустив всего одну ошибку.

3.4.4 Сравнительный анализ алгоритмов оптимизации

Для оценки влияния метода обновления весов на динамику и качество обучения был проведен сравнительный эксперимент: на одинаковой архитектуре сети (2 скрытых слоя по 24

нейрона, функция активации Sigmoid, 70 эпох) были обучены модели с 3 оптимизаторами: SGD, Momentum и Adam.

На рисунке 5 представлены сравнительные кривые функции потерь и точности на проверочной выборке.

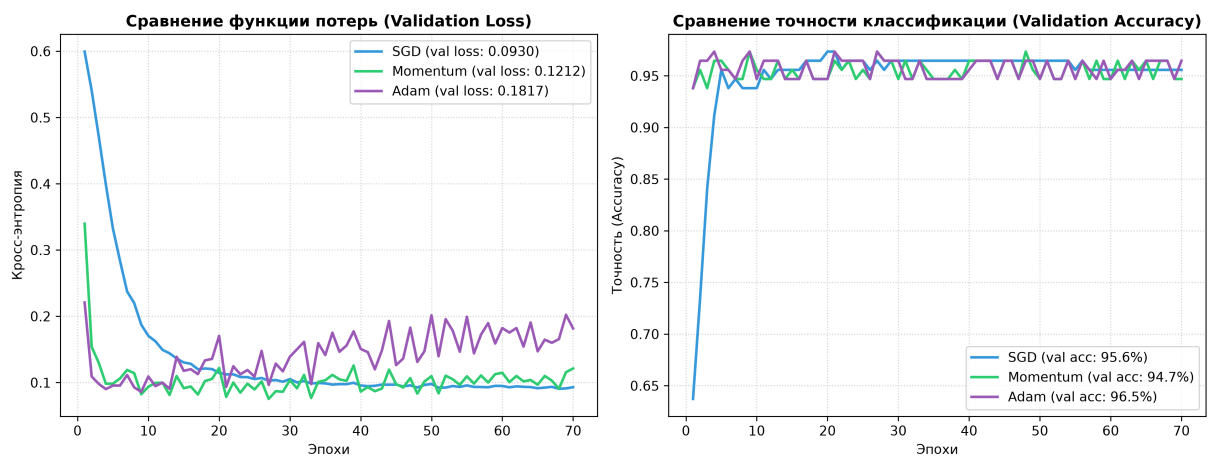


Рис. 5: Сравнение сходимости алгоритмов оптимизации (Validation Loss и Accuracy).

В таблице 2 приведены итоговые метрики каждого оптимизатора после 70 эпох обучения.

Таблица 2: Сравнение алгоритмов оптимизации на обучающей и проверочной выборках.

Оптимизатор	Train Loss	Validation Loss	Train Acc	Validation Acc
SGD	0.0494	0.0930	98.68%	95.58%
Momentum	0.0227	0.1212	99.12%	94.69%
Adam	0.0160	0.1817	99.78%	96.46%

Выводы по сравнению оптимизаторов:

- **SGD** продемонстрировал наиболее плавное и устойчивое снижение ошибки валидации (0.0930), показав лучшую обобщающую способность без склонности к переобучению;
- **Momentum** за счет накопления импульса быстрее преодолевает начальное плато и снижает ошибку на обучающих данных эффективнее базового SGD;
- **Adam** объединяет свойства адаптивного шага и инерции, благодаря чему достигает наивысшей точности на валидации (**96.46%**) существенно быстрее остальных методов.

4 Выводы

В ходе выполнения работы были достигнуты следующие результаты:

1. Проведен статистический и визуальный анализ набора данных Wisconsin Breast Cancer, выявлены ключевые информативные признаки (радиус, площадь, вогнутость ядер клеток).

2. Реализована схема предварительной обработки данных: заполнение пропусков по среднему значению соответствующего класса и Z -score стандартизация без утечки данных между выборками.
3. Разработан и протестирован многослойный персептрон на чистом NumPy с поддержкой модульного изменения числа слоев, нейронов, функций активации и оптимизаторов (SGD, Momentum, Adam).
4. Реализованы три независимые программы (`split.py`, `train.py`, `predict.py`), а также единый интерфейс `mlp.py` с возможностью гибкой настройки гиперпараметров сети через аргументы командной строки.
5. Проведен сравнительный анализ алгоритмов оптимизации: классический SGD обеспечил наилучшую устойчивость к переобучению (Val Loss = 0.0930), а метод Adam достиг наивысшей точности классификации на валидации (96.46%).
6. Итоговая модель показала точность 95.58% на проверочной выборке и полноту по злокачественным образованиям 97.62% при значении бинарной кросс-энтропии 0.0931, что полностью удовлетворяет требованиям задачи.